Arm® CCA RMM v1.1 Architecture Compliance Suite

Version 1.1

Validation Methodology

Confidential

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 02
107985 _0101_02_en

Arm® CCA RMM v1.1 Architecture Compliance Suite Validation Methodology

This document is Confidential. This document may only be used and distributed in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.
Arm only permits use of this document if you have reviewed and accepted Arm's Proprietary Notice found at the end of this document.

This document (107985 _0101_02_en) was issued on 2026-03-18. There might be a later issue at https://developer.arm.com/documentation/ 107985

The product version is 1.1.

See also: Proprietary Notice | Product and document information | Useful resources

Start reading

If you prefer, you can skip to the start of the content.

Intended audience

This document is intended for partners implementing the Arm Compute Subsystem.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

This document includes language that can be offensive. We will replace this language in a future issue of this document.

To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on https://support.developer.arm.com.

To provide feedback on the document, fill the following survey: https://developer.arm.com/documentation-feedback-survey.

Contents

1. Overview to CCA RMM ACS.... 4
1.1 Abbreviations....4
1.2 Realm Management Monitor....5
1.2.1 Realm Service Interface (RSI)....5
1.2.2 Realm Management Interface (RMI).... 6
1.2.3 Realm Host Interface (RHI).... 6
1.3 Architecture Compliance Suite.... 6
1.4 Directory structure.... 6

2. Validation Methodology.... 8
2.1 ACS software stack.... 8
2.2 Design under test.... 9
2.3 Test execution flow.... 9
2.4 ACS boot flow....11
2.5 Page table setup....14
2.6 ACS memory layout....14
2.7 Test report and analyzing results....16
2.8 Recovery from EL3/REL2 fault.... 17
2.9 MP execution setup....17

Proprietary Notice.... 18

Product and document information....20
Product status.... 20
Revision history....20
Conventions.... 21

Useful resources....24

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

1. Overview to CCA RMM ACS

This chapter introduces the features and components of Architecture Compliance Suite for Arm Realm Management Monitor (RMM).

1.1 Abbreviations

This section lists the abbreviations that are used in this document.

Table 1-1: Abbreviations and expansions

Abbreviation	Expansion
ABI	Application Binary Interface
ACS	Architecture Compliance Suite
API	Application Programming Interface
BSS	Block Started by Symbol
CCA	Confidential Compute Architecture
CPU	Central Processing Unit
EL	Exception Level
GOT	Global Offset Table
MMU	Memory Management Unit
MP	Multi-Processor
NS	Non-Secure
OSPM	Operating System Power Management
PAL	Platform Abstraction Layer
PSCI	Power State Coordination Interface
RHI	Realm Host Interface
RME	Realm Management Extension
RMI	Realm Management Interface
RMM	Realm Management Monitor
RSI	Realm Service Interface
SMC	Secure Monitor Call
SP	Secure Partition
SPM	Secure Partition Manager
SPMC	SPM Core
SPMD	SPM Dispatcher
SUT	System Under Test
UP	Uni-Processor
VAL	Validation Abstraction Layer
VM	Virtual Machine

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Abbreviation	Expansion
VMSA	Virtual Memory System Architecture
VPE	Virtual Processing Element

1.2 Realm Management Monitor

The Realm Management Monitor (RMM) is a software component that forms part of a system that implements the Arm Confidential Compute Architecture (Arm CCA). RMM is also responsible for the management of realm VMs and their interaction with the hypervisor in the normal world.

The RMM has following responsibilities in the Arm CCA system:

- Providing the services to the host.
- Allowing the host to manage the realms.

RMM also supplies services directly to the realms. The host services can be split into areas of policy and mechanics. For the policy functionality, the host owns the following policy decisions:

- When to create or destroy a realm.
- When to add or remove memory from a realm.
- When to schedule a realm.

The RMM supports the host policies by:

- Providing services to manipulate realm page tables - used in the creation or destruction and the addition or removal of realm memory.
- Managing realm context - save and restore used in scheduling interrupt support.
- PSCI call interception - there are power management requests. The RMM also provides services to realms, primarily attestation and cryptographic services.

The RMM also upholds the following security primitives for realms:

- Validates the host's requests for correctness.
- Isolates realms from each other.
- Helps in calculating the realm and platform attestation and measurements.

The RMM specification defines two communication channels to allow all functionalities to be requested and controlled between the Normal world Host and the realm VM.

1.2.1 Realm Service Interface (RSI)

A second channel defined between the RMM and the realm VM is called the RSI. The RSI is the channel for requesting services from the RMM. The RSI provides a channel for external services that allows few realm management operations to pass to the realm VM from the RMM. These

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

services can include cryptographic services and attestation. The RSI is also the channel for memory management requests from the realm VM to the RMM.

For more information on RMM, see the Realm Management Monitor specification.

1.2.2 Realm Management Interface (RMI)

The communication channel from the host to the RMM is called the RMI. The RMI allows the hypervisor to provide instructions to the RMM that manages the realm. The RMI uses SMC calls from the host hypervisor to request management control from the RMM. The RMI enables control of the realm management which includes the creation, population, execution, and destruction of the realms.

1.2.3 Realm Host Interface (RHI)

The Realm Host Interface (RHI) enables software running within realms to request specific services or data from the non-secure host.

This differs with the Realm Services Interface (RSI), which provides access to services or data from the Realm Management Monitor (RMM).

1.3 Architecture Compliance Suite

RMM Architecture Compliance Suite (ACS) contains a set of functional tests, demonstrating the invariant behaviors that are specified in the architecture specification. ACS tests are self-checking portable C and assembly-based tests with directed stimulus. The tests are open source and available with a BSD-3 license, allowing external contribution.

These tests are used to ensure the architecture compliance of the implementations to Arm RMM Architecture and they are not a substitute for design verification. ACS is expected to run on RMM and EL3 firmware stack on RME-enabled platforms.

For more information on tests, test steps, and test scenarios that are in scope and out of scope for compliance, see docs/testcase_checklist.md.

1.4 Directory structure

The following table describes the ACS components available in the top-level directory of the test suite when the release package is downloaded from GitHub:

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Figure 1-1: Directory structure of CCA RMM test suite

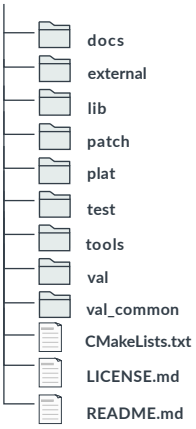


Table 1-2: Components and descriptions

Component	Description
docs	Contains the ACS documentation. Consists of Porting Guide, Test scenario document, and Validation Methodology document.
external	Contains external repositories cloned using git submodules.
lib	Contains build configuration files and repository-specific configuration headers for integrating external repositories.
plat	Contains files to form the PAL.
test	Contains self-checking C tests.
tools	Contains CMake files used to generate test binaries and run scripts to run tests on the reference platform.
val and val_common	Contains files to form the VAL.
CMakeLists.txt	Contains information on the CMake build support.
LICENSE.md	License information
README.md	Contains information on Arm CCA RMM ACS.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

2. Validation Methodology

This chapter describes the validation methodology for the Architecture Compliance Suite.

2.1 ACS software stack

This chapter describes the validation methodology for the Architecture Compliance Suite.

The ACS uses a layered software stack approach to enable porting across different test platforms. The constituents of the layered stack are:

- **Tests**

  This layer is a collection of targeted tests. It verifies if the target behavior conforms to the RMM specification. These tests map to one or more scenarios identified in the scenario document. They use interfaces that are provided by the VAL. A test does not directly interface with PAL functions. The test layer does not need any code modifications when porting from one platform to another.

- **VAL**

  This layer contains sub directories for the VAL libraries. It provides a uniform and consistent view of the available test infrastructure to the tests in the test pool. The VAL wraps RMI and RSI ABIs to exercise them in a generic way from the tests. The VAL also includes the PE initialization/boot code needed to run tests on the Host, Realm, and secure side. Any test dependency on platform information or functionality goes through via VAL functions.

- **PAL**

  This layer is the closest to the target and it must be aware of the required information about underlying hardware and firmware. Examples of platform-specific code include interactions with IO like UART. PAL must be ported or tailored to the platform on which ACS is run. For more information on porting setups refer, docs/porting_guide.md.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

2.2 Design under test

The following figure describes the design under test.

Figure 2-1: Design under test

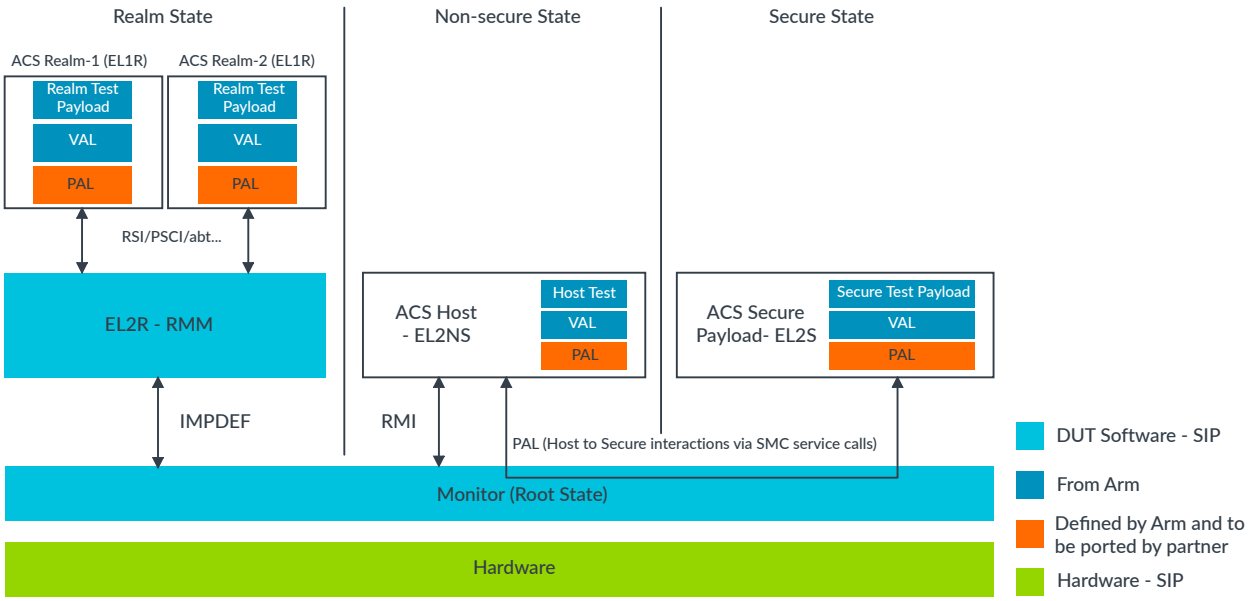


Table 2-1: Components and descriptions

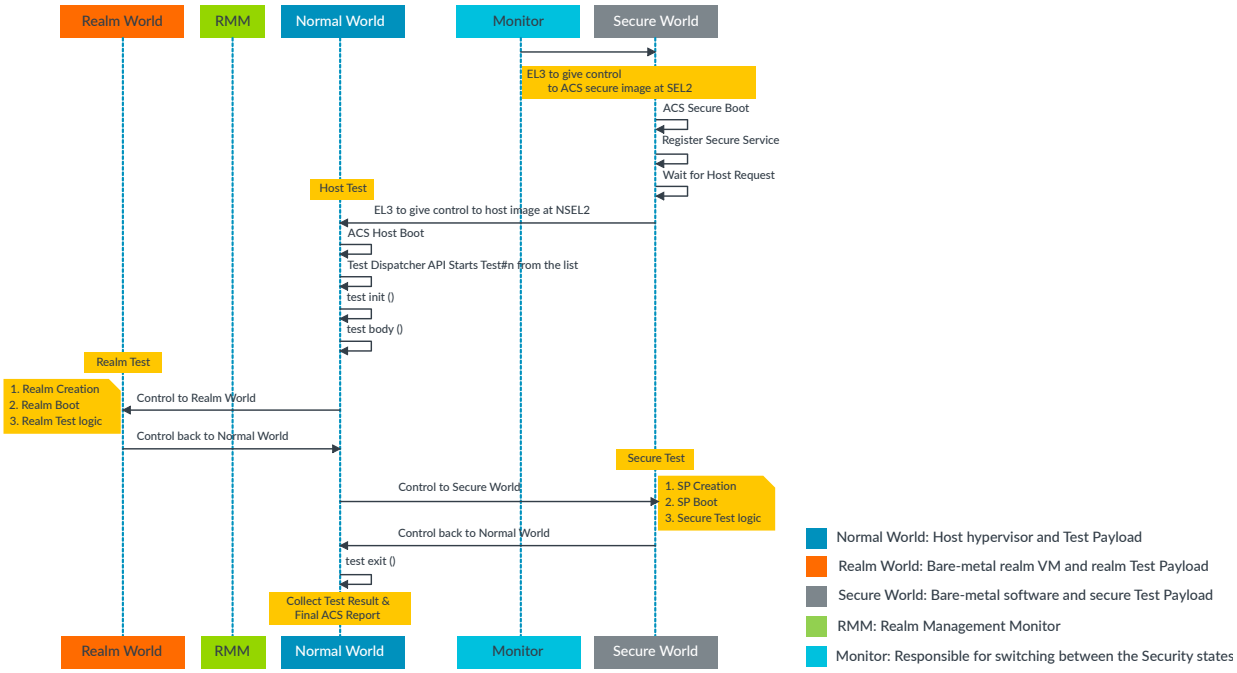
Component	Description
Hardware	Hardware system supporting Realm Management Extension (RME).
Monitor	The most privileged software component is responsible for switching between the security states. The monitor runs in the root state (EL3).
RMM	The software component is responsible for the management of Realms. RMM runs in the realm world at EL2.
ACS Host	ACS host is the bare-metal hypervisor containing the ACS host test payload. It is the anchor of the test. ACS host runs at EL2NS.
ACS Realm	ACS realm is the bare-metal realm VM containing the ACS realm test payload and it runs at EL1R. At runtime, the ACS host creates a realm VM by performing the necessary set of RMI commands. Interaction between the ACS host and realm code is through RMM-defined interfaces such as RMI calls, PSCI calls, and realm exit events.
ACS Secure	ACS secure is the bare-metal software containing ACS secure test payload and it can run at EL2S as SPMC. Interaction between the ACS host and secure code is through FF-A direct messaging interfaces over the PAL layer.

2.3 Test execution flow

The following figure describes the execution flow control of the compliance suite.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Figure 2-2: Test execution flow



The process that is followed for the flow control is:

1. System software component, like the Monitor loads ACS secure image (acs_secure.bin) in the secure memory.
2. Monitor gives control to the start of the image at EL2S.
3. ACS Secure code performs the necessary boot sequence.
4. ACS Secure code registers the service with Monitor and waits for the ACS host requests by giving control back to Monitor.
5. Monitor loads the combined image (acs_non_secure.bin) derived from ACS Host Image and ACS Realm image.
6. Monitor gives control to start the combined image at EL2NS.
7. ACS Host (anchor of the test regression) performs the necessary boot sequence and starts the test iteration loop.
8. Test entry and exits happen in the host.
9. Depending upon the test scenario, ACS Host Test will launch the ACS Realm in the Realm world and interact with ACS Realm Test or ACS Host Test will interact with ACS Secure Test.
10. Print status during the test execution as required.
11. Loop until all the checks of the test are completed. The test will exit the check loop on detecting check failure.
12. Loop until all the tests are completed.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Note

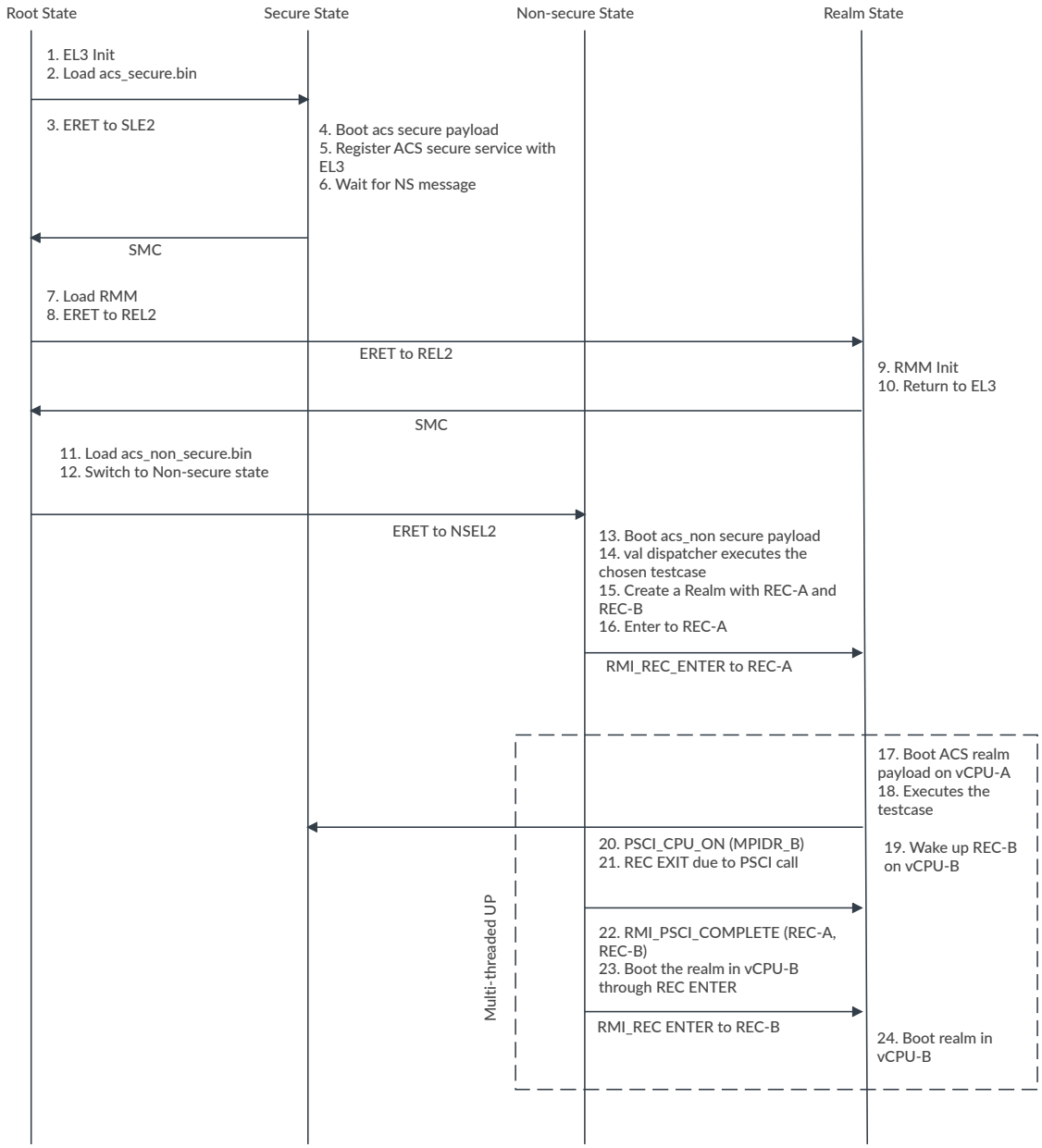
- Host test payload expects EL3 to complete the boot of Secure world and RMM. The order is immaterial from ACS PoV.
- With respect to the test, switching between Host and Realm is based on RMM specification.
- With respect to the test, switching between Host and Secure is through PAL. And current PAL implementation is based on FF-A Direct Messaging ABIs.

2.4 ACS boot flow

ACS has common boot flow sequences for Secure, Host, and Realm with each of them having a different entry point. The below diagram shows the boot sequence for the Host.

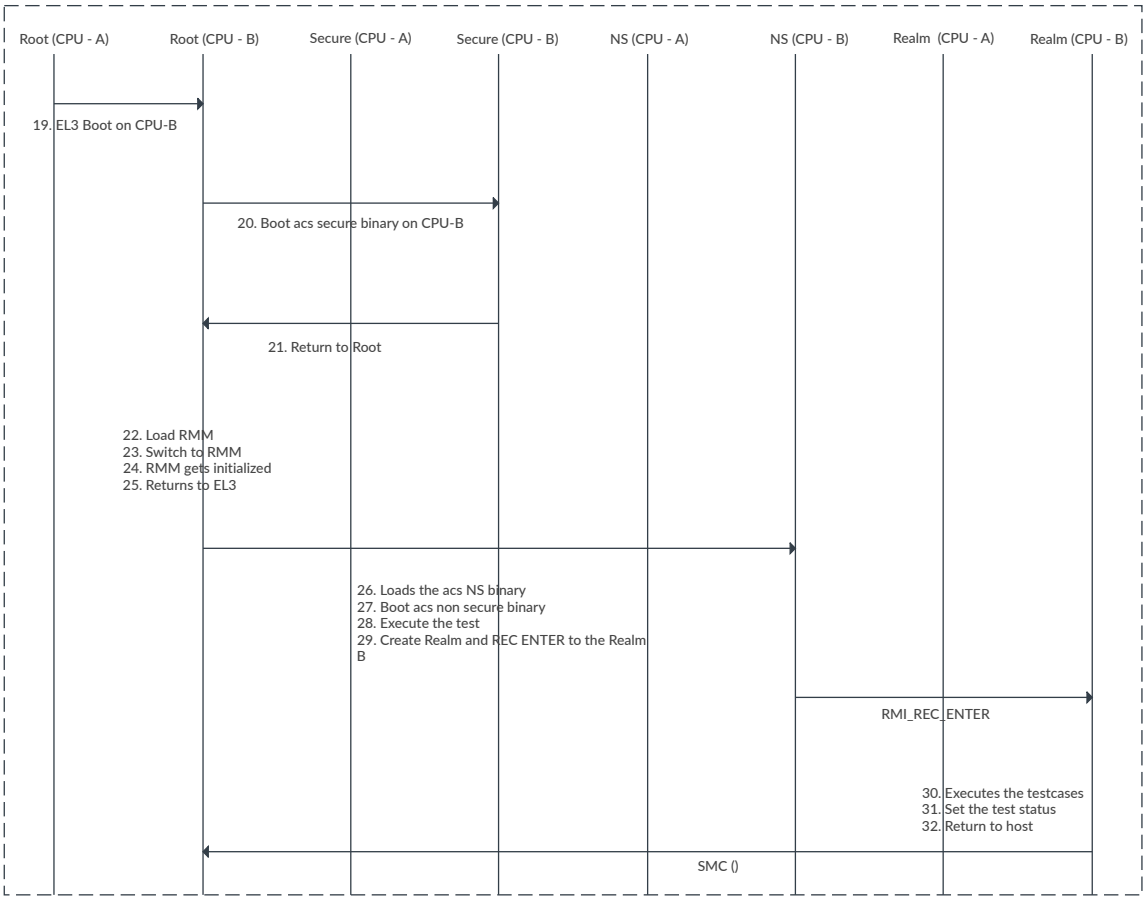
Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Figure 2-3: UP Realm with single or multi thread



Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Figure 2-4: True MP Realm



The following steps explain the boot flow sequence for the Host:

1. Program the system registers to set the vector table and enable the I-cache.
2. Invalidate the data cache and instruction cache for image regions to avoid the use of any stale cache data from previous boot stages.
3. Program the stack to enable C Programs.
4. Detect the Primary vs Secondary boot and save the primary CPU ID as leading CPU.

Primary boot:

1. Clear the BSS region.
2. Write the page table for Image regions, Device regions (Ex: UART), and NS Memory Pool.
3. Enable the MMU.
4. The host starts the test iteration loop.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Secondary boot:

1. Enable the MMU.
2. The host starts the test iteration loop.

For the Realm and Secure world, the implementation of the test dispatcher performs the following:

- It executes the dedicated Realm Test payload function for the test number shared by host.
- It executes the dedicated Secure Test payload function for the test number shared by host.
- The boot flow fixes the image symbols from the source view to the destination view before executing the BSS region init sequence.

2.5 Page table setup

ACS creates a page table for Image regions, Device regions (Ex, UART), and Memory Pool at runtime as part of the boot sequence.

- The size of image regions such as CODE, DATA, and BSS regions is calculated using symbols specified in the linker script.
- The size of Device regions is derived through the PAL.
- The size of the NS memory pool region is derived through PAL. NS memory pool is divided into two regions:
  - NS Heap Memory for Host use only. Host provisions memory resources such as RD, REC, RTT, and Data granules, to RMM and Test Realm out of this range.
  - NS region shared across Secure, Realm, and the Normal world. This region is used by VAL and Test to share information across all three worlds.

The memory for the page tables is reserved in the BSS segment based on the maximum table requirements. Boot enables the MMU with the below configuration:

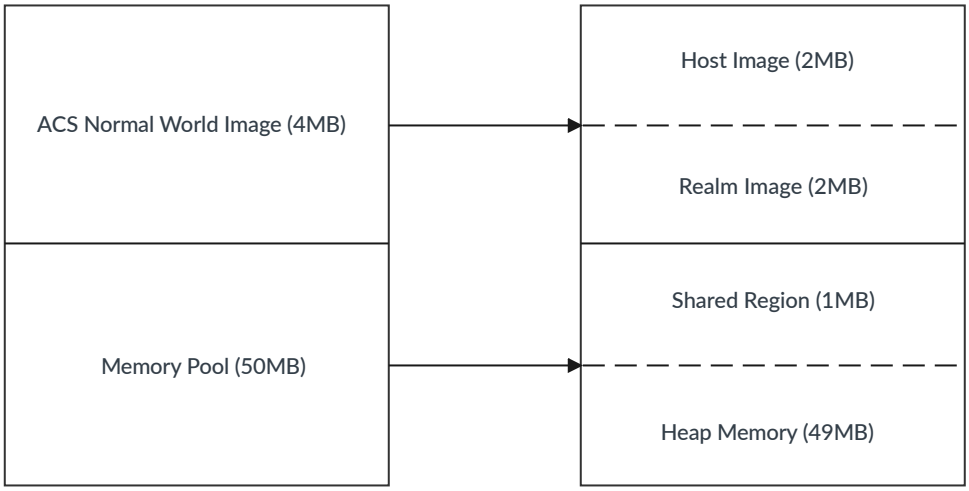
- 4KB Translation Granule.
- Support for non-flat map VA to PA is available to the test writer for memory under test. rest of the regions are flat mapped.

2.6 ACS memory layout

ACS requires the following memory regions that is provided by the platform:

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Figure 2-5: Memory layout



- NS Memory Pool (PAL configuration) - 52MB.
  - 4MB and 50MB regions of the NS memory pool must be contiguous but not as a whole 54MB.
  - First 4MB for loading NS host and initial Realm image. Reserve 2MB size for each image.
  - 49MB for Heap memory - Free NS range available to Host
- Host provisions below resources to RMM and Realm out of this range.
  - RDs, RTTs, RECs, Data Granules, Sharing of NS Granules to provide RD parameters, reference results, syncing threads in host and Realm, run objects, Granules for testing failure conditions.
- VAL provides dynamic allocation APIs to test to allocate memory from this pool.
  - 1MB of NS Shared Region.
- Information sharing across the world happens through this region. This region is mapped (stg1) across all worlds.
- S Memory Pool (PAL Configuration) - 1MB.
  - Memory reserve for Secure Test payload Image.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

2.7 Test report and analyzing results

When the ACS is run on the platform, each successful test must report either PASS, FAIL, SKIP or SIM ERROR. The following code is an example of a successful test pass.

```
***********************************

Suite=exception : Test=exception_rec_exit_wfi
HOST:
    Rec enter WFI Verified
Realm:
    REALM WFI Trigger checks are verified
Result=Passed

***********************************
Suite=exception : Test=exception_rec_exit_ripas
Realm:
    RIPAS Value RAM --> EMPTY verified
    RIPAS RAM --> EMPTY verified
    RIPAS Value RAM --> EMPTY verified with the PARTIAL RIPAS SET
    RIPAS Value RAM --> EMPTY verified with REJECT
Result=Passed

***********************************
```

The following code displays the regression report showing status of all the tests scheduled in a single execution run.

```
REGRESSION REPORT:
============================
   TOTAL TESTS     : 2
   TOTAL PASSED    : 2
   TOTAL FAILED    : 0
   TOTAL SKIPPED   : 0
   TOTAL SIM ERROR : 0
============================
******* END OF ACS *******
HOST: Entering standby..
```

If a test fails or skips, then you will see extra print messages to determine the cause.

Consider the following when debugging the failed test:

- Since each test is organized with a logical set of self-checking codes, if a failure occurs, searching for the relevant self-checking point is a useful point to start debugging.
    - Along with the error message, the test also prints the file and line number from where the error message is printed.
    - Test results contain the error code associated with the error message. The status of the error code is mapped with a structure val_status_t that is available at val/inc/val.h. Look for the enum that is dedicated to this number to see the status in the verbatim form.
- If the default prints does give enough information, you can recompile and rerun the test binaries with a high print verbosity level. See the test suite build README to understand how the test verbosity can be changed.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

2.8 Recovery from EL3/REL2 fault

Based on the test scenario, the test will perform a sequence to trigger expected fault conditions at EL3 or RMM level. For example, test triggering GPC fault. In such conditions, ACS needs platform support to redirect control to ACS Host or Realm to be able to conclude the current test results and execute the next test.

The following are the error handling steps if EL3/RMM supports the injection of faults into Host/Realm (abort injection from higher Exception Level (EL) to lower EL):

- Test installs the handler at the expected vector table location. For example, install a synchronous abort handler for stage 1 Data Abort.
- The test performs the authorized access and expects fault handling at EL3 or RMM.
- EL3/RMM injects the fault at lower EL by copying ESR.EC and FAR system registers values and updates ELR_ELx with VBAR+handlet_offset.
- Host/Realm receives the abort at the installed handler which concludes the error condition and returns to the interrupted code.

2.9 MP execution setup

On power-on reset, it is expected that ACS boots only on a single physical or Virtual PE (also called the primary PE) for ACS secure and ACS Host. ACS MP execution environment follows the <Power State Coordination Interface (PSCI)> specification.

The secondary execution context setup for the Host is as follows:

- The primary boot PE code uses the PSCI_CPU_ON interface to request the system software to initialize another PE of a given MPID and execute host code from the entry address specified during the PSCI_CPU_ON call.
- Handshaking and synchronization between Primary PE and secondary PEs are based on a spin-lock mechanism
- When the task on secondary PE is over, secondary PE uses the PSCI_CPU_OFF interface to exclude itself from the system.

The secondary execution context setup for Secure is as follows:

- Power management in the Secure world is as per FF-A specification. As described in the specification, the secure world does not perform power management independently from the Normal world. Instead, it is informed about OSPM operations initiated by the Normal world through PSCI functions.
- Secure code responses to the PSCI ON/OFF invocation from the Host and initialize the secondary PE.

MP execution setup for the Realm is followed as the RMM specification.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Proprietary Notice

This document is CONFIDENTIAL and any use by you is subject to the terms of the agreement between you and Arm Limited ("Arm") or the terms of the agreement between you and the party authorized by Arm to disclose this document to you.

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm. No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that, without obtaining Arm's prior written consent, you will not use or permit others to use the information: (i) for the purposes of determining whether the subject matter of this document infringes any third party patents; (ii) for developing technology or products which avoid any of Arm's intellectual property; (iii) as a reference for modifying existing patents or patent applications or creating any continuation, continuation in part, or extension of existing patents or patent applications; or (iv) for generating data for publication or disclosure to third parties, which compares the performance or functionality of the Arm technology described in this document with any other products created by you or a third party.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND

REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

If any of the provisions contained in these terms conflict with any of the provisions of any click through or signed written agreement covering this document with Arm, then the click through or signed written agreement prevails over and supersedes the conflicting provisions of these terms. This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at https://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1122-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is for an Alpha product, that is a product under development.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0101-02	18 March 2026	Non-Confidential	Alpha release for DA RMM v1.1 and Beta release for RHI v1.0
0101-01	4 September 2024	Non-Confidential	Beta release for Planes RMM v1.1
0100-02	15 July 2024	Non-Confidential	Alpha release for DA RMM v1.1
0100-01	11 December 2023	Non-Confidential	EAC release for RMMv1.0
0008-01	26 September 2023	Non-Confidential	Second release for v0.8
0007-01	31 March 2023	Non-Confidential	First release for v0.7

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in Document release information on page 20.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Table 2: Issue 0007-01

Change	Location
First release	-

Table 3: Differences between Issue 0007-01 and Issue 0008-01

Change	Location
Updated the code for the tests scheduled in a single execution run.	See, 2.7 Test report and analyzing results on page 15.

Table 4: Differences between Issue 0008-01 and Issue 0100-01

Change	Location
Updated the Page table setup description.	See, 2.5 Page table setup on page 14.

Table 5: Differences between Issue 0100-01 and Issue 0100-02

Change	Location
Updated the description of test report.	See, 2.7 Test report and analyzing results on page 15.
Updated abbreviation table.	See, 1.1 Abbreviations on page 4.

Table 6: Differences between Issue 0100-02 and Issue 0101-01

Change	Location
Updated the test report and analyzing results example code.	See, 2.7 Test report and analyzing results on page 15.

Table 7: Differences between Issue 0101-01 and Issue 0101-02

Change	Location
Updated abbreviation table.	See, 1.1 Abbreviations on page 4
Added a new section for Realm Host Interface.	See, 1.2.3 Realm Host Interface (RHI) on page 6
Updated the directory structure and the components description table.	See, 1.4 Directory structure on page 6
Updated the ACS memory layout figure and description.	See, 2.6 ACS memory layout on page 14
Updated the test report and analyzing results example code.	See, 2.7 Test report and analyzing results on page 15
Added a new Arm product resource.	See, Useful resources on page 24

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
italic	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
`monospace`	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
`monospace` <u>`underline`</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
`<and>`	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: `MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>`
SMALL CAPITALS	Terms that have specific technical meanings as defined in the *Arm® Glossary*. For example, IMPLEMENTATION DEFINED, IMPLEMENTATION SPECIFIC, UNKNOWN, and UNPREDICTABLE.

Caution

We recommend the following. If you do not follow these recommendations your system might not work.

Warning

Your system requires the following. If you do not follow these requirements your system will not work.

Danger

You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.

Note

This information is important and needs your attention.

Tip

This information might help you perform a task in an easier, better, or faster way.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Remember

This information reminds you of something important relating to the current content.

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Arm® Firmware Framework for Armv9-A	DEN0077A	Non-Confidential
Arm® SMC Calling Convention	ARM DEN 0028D	Non-Confidential
Arm® Architecture Reference Manual, for A-profile architecture	DDI0487	Non-Confidential
Arm® Power State Coordination Interface Platform Design Document	DEN0022F.b	Non-Confidential
Arm® Realm Management Monitor specification	DEN0137	Non-Confidential
Arm® Realm Host Interface specification	DEN0148	Non-Confidential

Arm architecture and specifications	Document ID	Confidentiality
Arm® Generic Interrupt Controller Architecture Specification GIC architecture version 3 and version 4	ID020922	Non-Confidential

Copyright © 2023–2024, 2026 Arm Limited (or its affiliates). All rights reserved.
Confidential